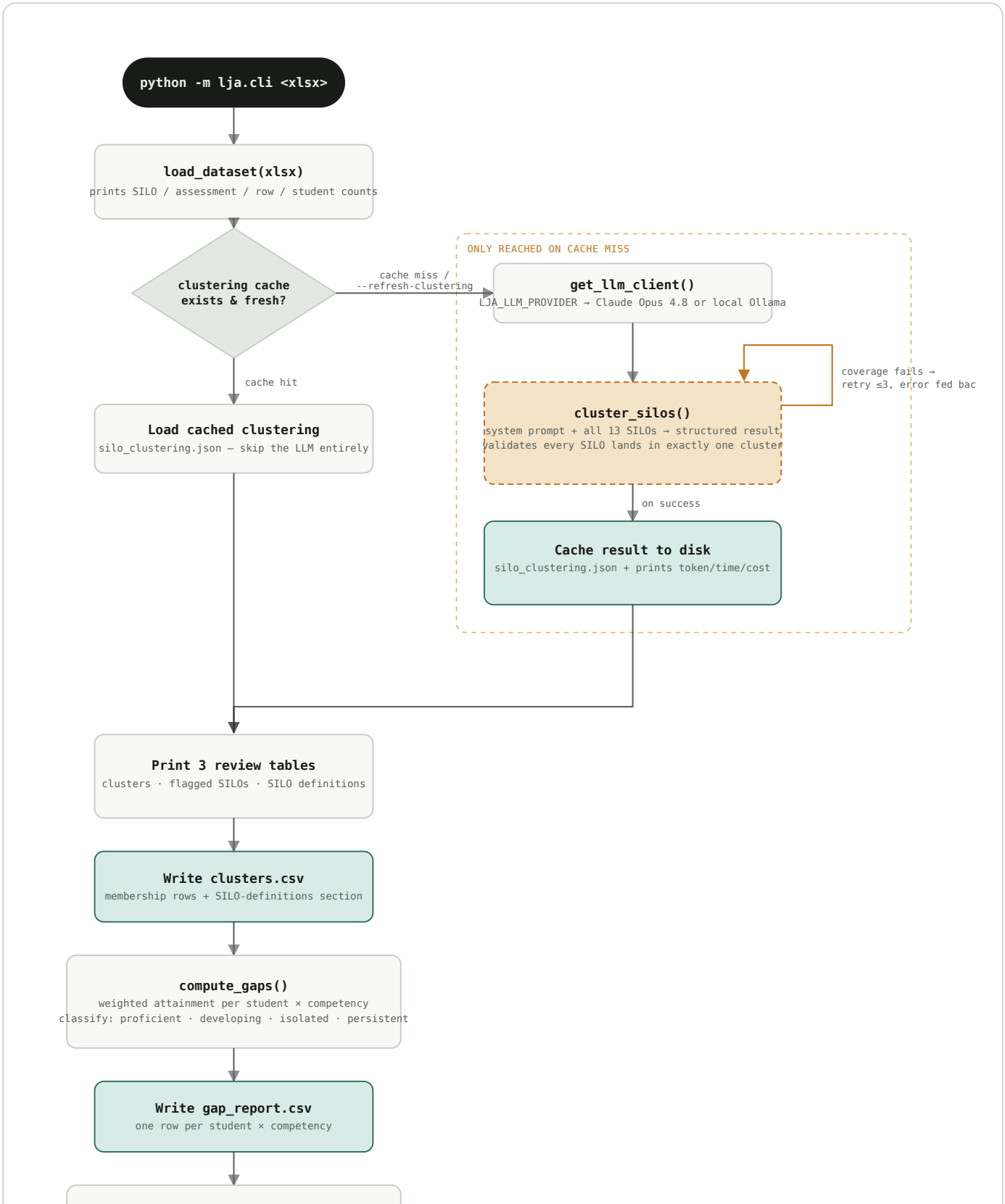
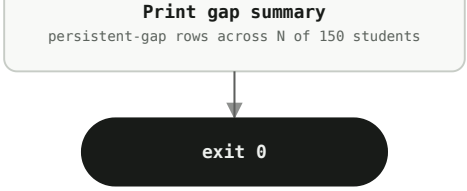
PYTHON -M LJA.CLI <XLSX>

LJA Pipeline

How the CLI turns Scott's Excel workbook into cross-subject competency clusters and a per-student gap report — with exactly one step that touches the network, cached so it only runs once.





in-process step (free, deterministic) branch network call — the only step that costs money

written to output/

The whole pipeline is local and free except for one step — `cluster_silos()` — which is gated behind a disk cache so a normal re-run never re-spends the LLM call, and which retries itself (not the whole pipeline) up to three times if the model drops or duplicates a SILO.

1	≤3	3
network call per run — cached after the first	clustering retries on a coverage failure	files written to output/

Reading the diagram

Cache hit vs. cache miss

Checked in `cli.py` before anything else runs: if `output/silo_clustering.json` already exists and `--refresh-clustering` wasn't passed, the LLM branch on the right is skipped entirely — every run after the first one is free.

The retry loop

`cluster_silos()` in `silo_clustering.py` checks that every one of the 13 SILOs landed in exactly one cluster. If the model drops or duplicates one, the next attempt's prompt names the exact SILOs that were wrong — up to three attempts before it gives up and raises.

Gap classification

`compute_gaps()` in `gap_detection.py` only classifies a competency as a *persistent gap* when attainment is low *and* at least two subjects provide evidence of it — a single bad assessment in one subject is an isolated gap, not a persistent one.

Reflects `python -m lja.cli` as implemented in `lja/cli.py`, `lja/model/silo_clustering.py`, and `lja/model/gap_detection.py`.